Programmation Python
Filières AD & SIE
S5
2025/2026

TP1

- **Objectif général du TP**

Développer une **mini-application modulaire** pour gérer un système de réservation de véhicules.

L'application doit être organisée sous forme de package Python, contenir une hiérarchie de classes, gérer un catalogue de véhicules, des clients, et un module de réservation.

- **Structure obligatoire du projet**

Le projet doit être organisé ainsi :

```
tp_reservation/
│
├── main.py
├── utils/
│   ├── __init__.py
│   └── outils.py
│
├── models/
│   ├── __init__.py
│   ├── vehicule.py
│   ├── voiture.py
│   ├── moto.py
│   ├── client.py
│   └── reservation.py
│
└── services/
```

```
├── __init__.py
└── gestion_reservations.py
```

Exercice 1 : Fonctions avancées & utilitaires

Fichier : utils/outils.py

1. Créez des fonctions utilitaires :
   - ***generer_id(prefix: str) -> str :*** utilise random, portée globale + locale, retourne par exemple : VEH-1032,
   - ***afficher_titre(titre: str, /, *, style="=", longueur=40) :*** arguments positionnels + keyword-only,
   - ***debug(*args, **kwargs) :*** utilise *args et **kwargs,
   - Une fonction d'ordre supérieur ***mesure_temps(func)*** retournant une version décorée de func.
2. Expliquez et montrez le comportement LEGB dans une fonction interne.

Exercice 2 : POO Base : Classe mère Vehicule

Fichier : models/vehicule.py

Créer une classe abstraite (pas besoin du module abc, juste un design logique) :

- Attributs :
  - *_marque* (protégé),
  - *_modele,*
  - *_prix_location,*
  - *id* (généré via la fonction utilitaire).
- Encapsulation + propriétés (@property, @setter),
- Constructeur secondaire via ***@classmethod from_string("Toyota;Yaris;150"),***
- Méthode d'instance : ***afficher_details(),***
- Méthode statique : ***est_vehicule_valide(marque),***
- Méthode polymorphe ***type_vehicule()*** à surcharger.

Exercice 3 : Héritage & Polymorphisme

Fichiers :

- models/voiture.py
- models/moto.py

1. Créez deux classes filles : **Voiture** et **Moto**

2. Ajoutez un attribut spécifique par classe (ex : nb_portes, cylindree)
3. Redéfinissez la méthode polymorphe ***type_vehicule(),***
4. Redéfinissez ***afficher_details()*** en appelant ***super(),***
5. Ajoutez des méthodes statiques pertinentes :
    - ***Voiture.est_suv(modele),***
    - ***Moto.est_sportive(cylindree).***

Exercice 4 : Composition : Client & Réservation

Fichiers :

- models/client.py
- models/reservation.py

➢ **Classe Client**

- Attributs : *nom*, *prenom*, *CIN*, *id* (généré),
- Méthode : ***afficher_profil().***

➢ **Classe Reservation**

- Contient **un client** et **un véhicule** (composition réelle)
- Attributs :
    - *client,*
    - *vehicule,*
    - *duree_jours,*
    - *coût total* (calcul automatique).
- Méthode : ***resume().***

Le véhicule n'existe pas en dehors d'une réservation ? Non, donc **composition partielle**.

Exercice 5 : Services & gestion des réservations

Fichier : services/gestion_reservations.py

- Créer une classe **GestionReservations** :
    - Liste interne des véhicules disponibles,
    - Liste interne des clients enregistrés,
    - Liste interne des réservations effectuées,
- Méthodes :
    - Ajouter un véhicule (***ajouter_vehicule***),
    - Ajouter un client (***ajouter_client***),

- Réserver un véhicule :
  - Vérifier si véhicule existant,
  - Vérifier si pas déjà réservé.
- Afficher le catalogue (polymorphisme → ***type_vehicule()***)
- Statistiques via méthodes de classe ou statiques :
  - Nombre total de réservations
  - Moyen de paiement le plus fréquent (utiliser kwargs dans réservation si voulu)

Exercice 6 : Programme principal

Fichier : main.py

- Créer un petit menu :

  1. Ajouter véhicule
  2. Ajouter client
  3. Réserver un véhicule
  4. Afficher réservations
  5. Quitter

- Tester :
  - Héritage,
  - Polymorphisme,
  - Encapsulation,
  - Modules/packages,
  - Fonctions avancées,
  - Composition.